

Player Rating Prediction — Project Report

1. Data Cleaning

The raw dataset (`train.csv` / `test.csv`) required three cleaning decisions before any modeling:

Step	What was done	Why
Drop unused-substitute rows	Removed all training rows where <code>minutes_played == 0</code>	42% of rows had <code>player_rating</code> forced to exactly <code>0.0</code> whenever <code>minutes_played == 0</code> . Keeping them would let the model "cheat" by learning <code>minutes==0 → 0</code> instead of real performance patterns. Test rows are never dropped (a prediction is required for every row); instead these rows are flagged (<code>zero_mask</code>) and their prediction is force-set to <code>0.0</code> directly, since the model never saw this pattern during training.
Drop ID/metadata columns	Removed <code>Id</code> , <code>player_id</code> , <code>player_name</code> , <code>team</code> , <code>match_id</code> , <code>match_date</code> , <code>stadium</code> , <code>city</code> , etc.	These identify a row rather than describe performance; they carry no generalizable signal.
One-hot encode categoricals	<code>position</code> , <code>preferred_foot</code> , <code>tournament_stage</code> , <code>match_result</code>	Standard encoding; train/test one-hot columns are reindexed against each other so both always align exactly, even if a category is rare in one split.

Feature selection: the `performance_score` decision

Two composite/derived score columns needed a judgment call: `performance_score` and `possession_impact`.

- `possession_impact` was dropped throughout — weak correlation with the target relative to other engineered scores, not clearly justified to keep.
- `performance_score` correlates **0.997** with `player_rating`, and a simple linear fit ($\text{rating} \approx 0.086 \times \text{performance_score} + 0.91$) explains nearly all of it. It's present in both `train.csv` and `test.csv`, so using it is **not technically leakage** (the information is genuinely available at prediction time) — but it's a derived summary of the same underlying stats, so using it shifts the task from "predict rating from raw match stats" toward "decode one column."

This was tested empirically rather than decided on principle alone: dropping it capped the MLP at RMSE 0.633; keeping it dropped RMSE to 0.279 — see the timeline below. It was ultimately **kept**, since nothing in the competition rules restricts feature choice, and the leaderboard is scored on raw MSE.

Engineered features

Five efficiency-ratio features were added to give the model cleaner signal than raw counts alone: `shot_conversion_rate`, `shot_accuracy`, `dribble_success_rate`, `cross_success_rate`, `aerial_duel_win_rate` (each a safe ratio, defaulting to 0 when the denominator is 0). Tested in isolation, these were **RMSE-neutral** on their own (0.6497 vs. 0.6495 baseline) but were kept since they cost nothing and matter more to tree models that split on individual features.

2. Model Improvement Timeline

All RMSE values below are on held-out validation data (or out-of-fold, where noted), tracked in the order they were built.

Phase 1 — without `performance_score` (leakage-free)

Step	Change	Val RMSE	R ²
1	Baseline MLP, bug-fixed (feature scaling added, device handling fixed,	0.671	0.175

Step	Change	Val RMSE	R ²
	broken "accuracy" metric replaced with RMSE)		
2	+ BatchNorm + Dropout	0.650	0.226
3	+ ReduceLROnPlateau scheduler + early stopping	0.633	0.265
4	XGBoost (same cleaned features)	0.618	0.299
5	Ensemble (weighted blend of MLP + XGBoost)	0.618	0.300

Attempted but reverted: tail-weighted loss (upweighting rows far from the mean rating, to fix systematic over-/under-prediction at rating extremes) made results *worse* at every weight tested ($\alpha = 0.1, 0.3, 1.0$) — the extreme-rating buckets (ratings 0-4, 8-10) had only 6 and 56 validation rows respectively, too few to weight reliably without destabilizing the well-populated middle.

Phase 2 — with **performance_score** included

Step	Change	Val RMSE	MSE
6	MLP retrained with performance_score (single 80/20 split)	0.2798	0.0783
7	XGBoost retrained with performance_score (single split)	0.2746	0.0754
8	Ensemble blend (30% MLP / 70% XGBoost)	0.2740	0.0751

Step	Change	Val RMSE	MSE
9	MLP, proper 5-fold cross-validation (out-of-fold RMSE)	0.2829	0.0800
10	XGBoost, proper 5-fold cross-validation (out-of-fold RMSE)	0.2718	0.0739

A key finding surfaced here: fitting `player_rating` on `performance_score` **alone**, with plain linear regression, gives a residual standard deviation of **0.2770** — almost identical to every full-feature model above. This means the ~70 other features contribute only marginal improvement (≈ 0.005 RMSE) beyond what one column already explains; the models are near the irreducible noise floor of this target given the available features, not underfitting.

Bugs fixed along the way

Several correctness issues were caught and fixed during development, independent of architecture choices:

- Missing feature scaling (caused loss to reach ~5.7 billion on the first training step before slowly recovering)
- Prediction tensors never moved to `device`, and `.numpy()` called on GPU tensors (crashes when CUDA is available)
- `best_state = model.state_dict()` not cloned — silently saved the *final* epoch's weights instead of the actual best one, defeating early stopping
- A broadcasting bug (`.squeeze()`-ed predictions of shape `(64,)` compared against targets of shape `(64, 1)`) silently computing loss over a `(64, 64)` matrix instead of 64 correctly paired values
- A classification-style `accuracy()` function (thresholding continuous ratings at 0.5) that was always ≈ 0 and meaningless for this regression task — replaced with RMSE/R², matching the competition's actual MSE-based evaluation
- Submission column name mismatch (`Player ID` vs. the `Id` Kaggle's checker actually expected)

3. XGBoost as an Add-On to the MLP

XGBoost was introduced alongside the MLP, then combined via ensembling, for three reasons:

1. **Tree splits handle rare, high-leverage events better.** A hat-trick, a red card, or a shutout are rare but rating-defining events. Trees can carve out a clean split for these directly; a small MLP tends to smooth them away, regressing predictions toward the mean (a pattern confirmed in error analysis — the MLP under-predicted great games and over-predicted poor ones).
2. **No feature scaling required**, and comparable or better accuracy with less tuning — XGBoost consistently matched or beat the MLP at every stage (e.g., 0.2718 vs. 0.2829 out-of-fold).
3. **Ensembling exploits different error patterns.** The two model families make different mistakes on different rows, so even a simple weighted average of their predictions outperforms either alone; a learned (stacked) blend — fitting a linear meta-model on out-of-fold predictions from both — refines this further by finding the optimal combination weights automatically instead of guessing a fixed ratio.

Final approach: train both models with identical 5-fold cross-validation splits, average each model's predictions across its 5 folds (a free mini-ensemble within each model), then blend the two models' out-of-fold predictions — either via a fixed weighted average or a learned linear stacking model — to produce the final submission.

4. Summary

Metric	Leakage-free best	Final best (with <code>performance_score</code>)
Val/OOF RMSE	0.618 (ensemble)	0.2718 (XGBoost, 5-fold CV)
Val/OOF MSE	0.382	0.0739

The largest single jump in accuracy came from the `performance_score` feature decision, not from architecture or hyperparameter tuning — a useful lesson in checking feature-target correlations early. Beyond that point, further gains (ensembling, cross-validation, stacking) were real but incremental, consistent with the model sitting close to the dataset's inherent noise floor.

References

- Paszke, A. et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. pytorch.org/docs

- Chen, T. & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. KDD '16. xgboost.readthedocs.io
- Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. JMLR 12. Used for **KFold** and **LinearRegression** stacking. scikit-learn.org/stable/documentation.html